CSCI2160Quiz6F11SubProcedures Name: Brandon Buchanan

93

1) Suppose that you have the below data segment to work with.

0000 bVal1 BYTE "ONCE UPON A MIDNIGHT DREARY",0

001C bVal2 BYTE 20 dup(?)

0030 wVal1 WORD 8,19,27,32,-20

003A wVal2 WORD 23,55,66,77,22,-13,-15

0048 dVal1 DWORD 87,92,14,25

005C dVal2 DWORD 5 dup(?)

Complete the below table:

Location Counter	**Identifier**	TYPE	LENGTHOF	SIZEOF
0000	bVal1	1	28	28
001C	bVal2	1	20	20
0030	wVal1	2	5	10
003A	wVal2	2	7	14
0048	dVal1	4	4	16
~~005C~~ 58	dVal2	4	5	20

-1

-1/23 * 10

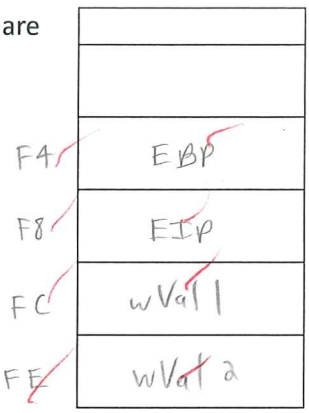
2) For the problems on the following page, write the values of the stack pointer on the left, and assuming that the first instructions within the procedure are **push ebp** and **mov ebp,esp**. Draw the stack as it would appear AFTER executing those two instructions within the procedure. Assume also that the prefix of the variable indicates whether it is a **word, dword, byte**. Also, assume the use of the **&** in front of a variable, assumes that the address of the variable is what is used as a parameter.

-0

Write the calling sequence for each of the below. You may assume, that the parameters are correct with the type and order as they appear:

a) ;dVal1 = f1(wVal1, wVal2,)

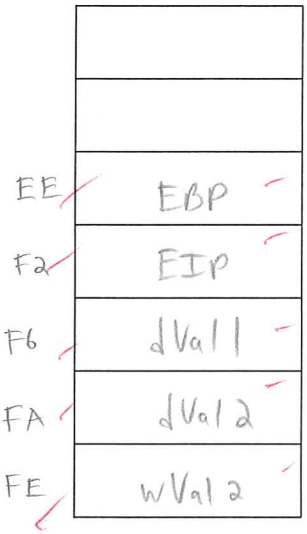
```
push wVal2
push wVal
call f1
mov dVal, eax
add esp, 4
```



100----->

b) ;wVal1 = f2(dVal1, dVal2,wVal2)

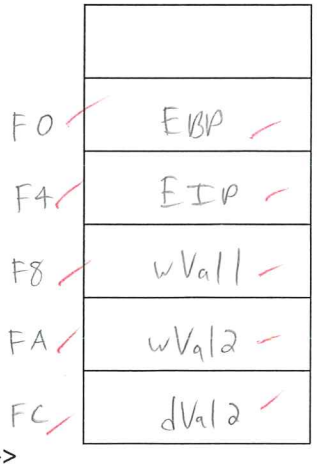
```
push wVal2
push dVal2
push dVal1
call f2
mov wVal1, ax
add esp, 10
```



100----->

c) ; f3(wVal1, wVal2,dVal2)

```
push dVal2
push wVal2
push wVal
call f3
add esp, 8
```



100----->

-0

3) Given the below UML statements for the methods, and given that the prefix indicates whether the variable is a BYTE (byte), WORD (short), or DWORD (int), Write the call sequence. Remember that the variable types that are passed MUST match what the prototype requires. For example, if you pass a byte as a parameter that expects a word, then you MUST properly prepare that parameter before you pass it. This is what happens when variables are "cast" in Java. I am giving you the UML statement for the java method and gave you an example for the first one.

f1 (int, short, short):int
f2(short, short):void
f3(int, short, int):short
f4(int,int,int):void
f5(short, short, int):int

a) iNum = f1(iVal, sNum, bVal); //note that you want to pass a byte where a short is expected. You would be required to rewrite this in Java as defined below. That is because Java will NOT allow you to alter the type of an expected parameter unless you force it.

```
;       iNum = f1(iVal, sNum, (short)bVal);
; The generated sequence of assembly language instructions would be:
        mov     al,bVal
        cbw
        push    ax
        push    sNum
        push    iVal
        call    f1
        mov     iNum, eax
        add     esp, 8
```

b) IVal = f1(iNum1, iNum2, sVal);

```
push sVal
mov eax, iNum2
push ax
push iNum1
call f1
mov IVal, qword ptr eax
add esp, 8
```

```
cdq
mov ebx, offset lval
mov [ebx+4], edx
mov [ebx], eax
```

c) iNum = f1(iNum1, iNum2, iNum3);

```
mov eax, iNum3
push ax
mov eax, iNum2
push ax
push iNum1
call f1
mov iNum, eax
add esp, 8
```

-0

f1 (int, short, short):int
f2(short, short):void
f3(int, short, int):short
f4(int,int,int):void
f5(short, short, int):int

d) f2(sVal1, iVal2);

```
mov eax, iVal2
push ax
push sVal1
call f2
add esp, 4
```

e) iNum = f3(sVal1, sVal2, sVal3);

```
mov ax, sVal3
cwde
push eax
push sVal2
mov ax, sVal1
cwde
push eax
call f3
mov iNum, dword ptr ax
add esp, 10
```

cwde
mov iNum, eax

f) f4(iVal, bVal, sVal);

```
mov ax, sVal
cwde
push eax
mov al, bVal
cbw
cdwe
push eax
push iVal
call f4
add esp, 12
```

f1 (int, short, short):int
f2(short, short):void
f3(int, short, int):short
f4(int,int,int):void
f5(short, short, int):int

g) iNum = f5(sVal1, iVal1, sVal2);

```
mov ax, sVal2
cwde
push eax
mov eax, iVal1
push ax
push sVal1
call f5
mov iNum, eax
add esp, 8
```

h) f5(sVal1, sVal2, iVal3);

```
push iVal3
push sVal2
push sVal
call f5
add esp, 8
```

4) Given the below java statements where Hungarian notation is used (prefix indicates the type), write the corresponding assembly instructions that could be generated to produce the desired result:

a) iVal1 = sVal1 + sVal2;

```
mov eax, dword ptr sVal1
add eax, dword ptr sVal2
mov iVal1, eax
```

-4

```
mov ax, sVal2
ADD ax, sVal1
cwde
mov iVal1, eax
```

b) iVal2 = sVal2 + iVal1;

~~mov eax, dword ptr sVal2~~

```
add eax, iVal
mov iVal2, eax
```

movsx eax, sVal2

-1

c) sVal = bVal1 * bVal2;

```
mov al, bVal1
imul bVal2          ; ax
mov sVal, ax
```

d) sVal3 = sVal1 * sVal2;

```
mov ax, sVal1
imul sVal2          ; dx:ax
mov sVal, ax        ; dx is lost
```

ok

e) iVal1 = sVal1 * sVal2;

```
mov ax, sVal
imul sVal2          ; dx:ax
mov ebx, offset iVal
mov [ebx], dx
mov [ebx+2], ax
```

-2

f) iNum3 = iNum2 * iNum3;

```
mov eax, iNum2
imul iNum3          ; edx:eax
mov iNum3, eax
```

g) lVal = iNum1 * iNum2; (long Val)

```
mov eax, iNum1
imul iNum2          ; edx:eax
mov ebx, offset lVal
mov [ebx], edx
mov [ebx+4], eax
```

-2